



Least Authority
PRIVACY MATTERS

SHA-3

Security Audit Report

Midnight Foundation

Final Audit Report: 23 February 2026

Table of Contents

[Overview](#)

[Background](#)

[Project Dates](#)

[Review Team](#)

[Coverage](#)

[Target Code and Revision](#)

[Supporting Documentation](#)

[Areas of Concern](#)

[Findings](#)

[General Comments](#)

[System Design](#)

[Code Quality](#)

[Documentation and Code Comments](#)

[Scope](#)

[Specific Issues & Suggestions](#)

[Issue A: Denial of Service \(DoS\) via Off-By-One in Permutation Row Count](#)

[Suggestions](#)

[Suggestion 1: Enforce Field Size Assumption In-Circuit](#)

[Suggestion 2: Replace Debug Assertion](#)

[Suggestion 3: Automate Bootstrapping](#)

[Suggestion 4: Improve Test Coverage](#)

[About Least Authority](#)

[Our Methodology](#)

Overview

Background

Midnight Foundation has requested Least Authority perform a security audit of the SHA-3 circuit.

Project Dates

- **January 16, 2026 - January 28, 2026:** Initial Code Review (*Completed*)
- **January 29, 2026:** Delivery of Initial Audit Report (*Completed*)
- **February 23, 2026::** Verification Review (*Completed*)
- **February 23, 2026:** Delivery of Final Audit Report (*Completed*)

Review Team

- George Gkitsas, Security / Cryptography Researcher and Engineer
- Eduardo Morais, Cryptography Researcher and Engineer
- Burak Atasoy, Project Manager
- Jessy Bissal, Technical Editor

Coverage

Target Code and Revision

For this audit, we performed research, investigation, and review of the SHA-3 circuit and its instantiation followed by issue reporting, along with mitigation and remediation instructions as outlined in this report.

The following code repository is considered in scope for the review:

- Keccak / SHA-3:
<https://github.com/alexandroszacharakis8/sha3-circuit>

Specifically, we examined the following Git revision for our initial review:

- `d75f363aa1f8e43031823cd17271e091d344a5ef`

For the verification, we examined the following Git revision:

- `12addf950bb17ee7a76389c60e5f61113733fdf2`

For the review, these repositories were cloned for use during the audit and for reference in this report:

- sha3-circuit:
<https://github.com/LeastAuthority/sha3-circuit>
- midnight-zk:
https://github.com/midnightntwrk/midnight-zk/blob/3aa589bde60973745552b24dba9ea480fb007ffb/zk_stdlib/src/external/keccak_sha3.rs

All file references in this document use Unix-style paths relative to the project's root directory.

In addition, any dependency and third-party code, unless specifically mentioned as in scope, were considered out of scope for this review.

Supporting Documentation

The following documentation was available to the review team:

- Websites:
 - <https://midnight.network>
 - <https://shielded.io>

In addition, this audit report references the following documents:

- Keccak Specification Summary:
https://keccak.team/keccak_specs_summary.html
- The halo2 Book:
<https://zcash.github.io/halo2>

Areas of Concern

Our investigation focused on the following areas:

- Correctness of the implementation;
- Vulnerabilities within each component and whether the interaction between the components is secure;
- Whether requests are passed correctly to the network core;
- Key management, including secure private key storage and management of encryption and signing keys;
- Denial of Service (DoS) and other security exploits that would impact the intended use or disrupt the execution;
- Protection against malicious attacks and other ways to exploit;
- Inappropriate permissions and excess authority;
- Data privacy, data leaking, and information integrity; and
- Anything else as identified during the initial analysis phase.

Findings

General Comments

This project is a Halo2 circuit implementation of the Keccak-f permutation and gadgets for SHA3-256 and Keccak256 hashing. It uses a single-lookup table approach with packed arithmetic techniques to build efficient constraints.

We checked the correctness of the implementation against the specification, reviewed circuit constraints and test coverage for circuit soundness, and assessed other security attack vectors. We identified a denial-of-service issue that can be triggered by an attacker controlling the message length ([Issue A](#)).

We also reviewed the instantiation of the Keccak256 circuit within the midnight-zk project. We assessed whether assumptions about inputs are met and verified that the circuit's instantiation does not affect its soundness.

System Design

Our team examined the SHA-3 circuit and its instantiation and found the system to be well-designed, with tight circuit constraints and a strong focus on security. The documentation is clear and helpful, and we recommend maintaining the current standard going forward.

Dependencies

Our team did not identify any vulnerabilities in the implementation's use of dependencies.

Code Quality

We performed a manual review of the repositories in scope and found the code to be well organized and aligned with development best practices.

Tests

The SHA-3 circuit implementation demonstrates strong overall test coverage for core arithmetic operations and end-to-end functionality, including a simple mechanism for negative tests to validate that invalid proofs fail as expected across many components. However, we note opportunities to strengthen coverage through fuzzing and property-based testing, and more rigorous validation of padding checks ([Suggestion 4](#)).

Documentation and Code Comments

The project documentation clearly outlines the project's purpose and sufficiently describes the system's intended functionality. Additionally, the codebase includes descriptive comments, which aid in understanding the intended behavior of the relevant components.

Scope

The scope of this review was sufficient and included all security-critical components.

Specific Issues & Suggestions

We list the issues and suggestions found during the review, in the order we reported them. In most cases, remediation of an issue is preferable, but mitigation is suggested as another option for cases where a trade-off could be required.

ISSUE / SUGGESTION	SEVERITY	STATUS
Issue A: Denial of Service (DoS) via Off-By-One in Permutation Row Count	Low	Resolved
Suggestion 1: Enforce Field Size Assumption In-Circuit	Informational	Resolved
Suggestion 2: Replace Debug Assertion	Informational	Resolved
Suggestion 3: Automate Bootstrapping	Informational	Resolved
Suggestion 4: Improve Test Coverage	Informational	Unresolved

Issue A: Denial of Service (DoS) via Off-By-One in Permutation Row Count

Location

[packed_chip/keccakf_operations/mod.rs#L49](#)

[packed_chip/keccakf_operations/compute_iota.rs#L52](#)

[packed_chip/keccakf_operations/compute_iota.rs#L74](#)

[src/packed_chip/mod.rs#L653](#)

Synopsis

Off-by-one in permutation row count can cause `min_k` to choose a too-small `k`, causing proving to fail when the final `iota` step writes one extra row.

Impact

Low.

An attacker can cause proof generation to fail deterministically for specific lengths when the service sizes parameters using `min_k`, resulting in denial of service for those requests.

Feasibility

Medium.

Severity

Low.

Preconditions

An attacker must be able to define or influence the proving message length.

Technical Details

`compute_last_iota` uses a two-row layout. It calls `assign_bootstrap2` at `LAST_IOTA_OFFSET`, which writes rows `LAST_IOTA_OFFSET` and `LAST_IOTA_OFFSET + 1`, and then assigns constants on the second row. However, `KECCAK_ROWS_PER_PERMUTATION` is defined as `LAST_IOTA_OFFSET + 1`, which treats the first `iota` row as the permutation's last row. `min_k` relies on that constant to size the circuit, so it can select a `k` that fits the underestimated layout but is one row too small for the actual layout. When synthesis reaches the final `iota` step, it attempts to write the extra row and fails.

Remediation

We recommend changing `KECCAK_ROWS_PER_PERMUTATION` to `LAST_IOTA_OFFSET + 2` [here](#).

In addition, for consistency, we recommend updating the comment [here](#) to `"KECCAK_ROWS_PER_PERMUTATION rows per permutation."`

Status

The Midnight team has [resolved](#) the issue as recommended.

Verification

Resolved.

Suggestions

Suggestion 1: Enforce Field Size Assumption In-Circuit

Location

[src/packed_chip/mod.rs#L346](#)

[src/packed_chip/utils/mod.rs#L123](#)

[packed_chip/utils/mod.rs#L172](#)

[packed_chip/bytes_to_spread/mod.rs#L34](#)

Synopsis

Missing field size constraints can render Keccak-f constraints unsound due to modular wraparound.

Specifically, the circuit relies on an injective embedding of 64-bit Keccak lanes into a 192-bit spread representation and on non-wrapping linear recompositions, but it is generic over $F: \text{PrimeField}$. It does not enforce that the field modulus is large enough, so for sufficiently small fields the same field element can represent multiple different 192-bit spread values and invalid Keccak states can satisfy constraints.

Mitigation

Minimum size assumption is [stated in the documentation](#). We recommend adding checks to enforce it.

Status

The Midnight team has [added](#) a runtime check.

Verification

Resolved.

Suggestion 2: Replace Debug Assertion

Location

[packed_chip/keccakf_operations/compute_chi.rs#L86](#)

Synopsis

The function contains a `debug_assert!(round == KECCAK_NUM_ROUNDS - 1)` that validates message absorption only on the final Keccak round. This is a critical security constraint for correct protocol execution, but `debug_assert!` statements are compiled out in release builds. As a result, this validation would not occur in production deployments.

Mitigation

We recommend replacing the `debug_assert!` with a runtime check that returns an `Error` in release mode. Accordingly, the following statement should be updated:

```
debug_assert!(round == KECCAK_NUM_ROUNDS - 1);
```

Status

The Midnight team stated that the `debug_assert` is intended solely as a sanity check during development. Accordingly, the team does not consider this to be an issue.

Verification

Resolved.

Suggestion 3: Automate Bootstrapping

Location

[src/packed_chip/bootstrap](#)

Synopsis

Bootstrapping must be invoked manually to correct errors that occur after binary operations. This approach is error-prone to implement and review.

Mitigation

We recommend implementing a mechanism that allows bootstrapping to be executed automatically.

Status

The Midnight team stated that this operation is called via API.

Verification

Resolved.

Suggestion 4: Improve Test Coverage

Location

Across the codebase.

Synopsis

Attackers can potentially exploit underconstrained circuits. Preventing this type of security vulnerability becomes increasingly difficult as circuits grow in size and complexity.

Mitigation

We recommend improving test coverage by adding more complex negative test cases. Techniques such as fuzzing and property-based testing can increase confidence in circuit soundness.

Status

The Midnight team has acknowledged this suggestion. However, no updates regarding test coverage were available at the time of verification.

Verification

Unresolved.

About Least Authority

We believe that people have a fundamental right to privacy and that the use of secure solutions enables people to more freely use the Internet and other connected technologies. We provide security consulting services to help others make their solutions more resistant to unauthorized access to data and unintended manipulation of the system. We support teams from the design phase through the production launch and after.

The Least Authority team has skills for reviewing code in multiple Languages, such as C, C++, Python, Haskell, Rust, Node.js, Solidity, Go, JavaScript, ZoKrates, and circom, for common security vulnerabilities and specific attack vectors. The team has reviewed implementations of cryptographic protocols and distributed system architecture in cryptocurrency, blockchains, payments, smart contracts, zero-knowledge protocols, and consensus protocols. Additionally, the team can utilize various tools to scan code and networks and build custom tools as necessary.

Least Authority was formed in 2011 to create and further empower freedom-compatible technologies. We moved the company to Berlin in 2016 and continue to expand our efforts. We are an international team that believes we can have a significant impact on the world by being transparent and open about the work we do.

For more information about our security consulting, please visit <https://leastauthority.com/security-consulting/>.

Our Methodology

We like to work with a transparent process and make our reviews a collaborative effort. The goals of our security audits are to improve the quality of systems we review and aim for sufficient remediation to help protect users. The following is the methodology we use in our security audit process.

Manual Code Review

In manually reviewing all of the code, we look for any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behavior when it is relevant to a particular line of investigation.

Vulnerability Analysis

Our audit techniques include manual code analysis, user interface interaction, and whitebox penetration testing. We look at the project's website to get a high level understanding of what functionality the software under review provides. We then meet with the developers to gain an appreciation of their vision of the software. We install and use the relevant software, exploring the user interactions and roles. As we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation. We hypothesize what vulnerabilities may be present and possibly resulting in Issue entries, then for each, we follow the following Issue Investigation and Remediation process.

Documenting Results

We follow a conservative and transparent process for analyzing potential security vulnerabilities and seeing them through successful remediation. Whenever a potential issue is discovered, we immediately create an Issue entry for it in this document, even before having verified the feasibility and impact of the issue. This process is conservative because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyze the feasibility of an attack in a live system.

Suggested Solutions

We search for immediate and comprehensive mitigations that live deployments can take, and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinized by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our Initial Audit Report, and before we perform a verification review.

Before our report, including any details about our findings and the solutions are shared, we like to work with your team to find reasonable outcomes that can be addressed as soon as possible without an overly negative impact on pre-existing plans. Although the handling of issues must be done on a case-by-case basis, we always like to agree on a timeline for a resolution that balances the impact on the users and the needs of your project team.

Resolutions & Publishing

Once the findings are comprehensively addressed, we complete a verification review to assess that the issues and suggestions are sufficiently addressed. When this analysis is completed, we update the report and provide a Final Audit Report that can be published in whole. If there are critical unaddressed issues, we suggest the report not be published and the users and other stakeholders be alerted of the impact. We encourage that all findings be dealt with and the Final Audit Report be shared publicly for the transparency of efforts and the advancement of security learnings within the industry.